

0.1 Môi trường và dữ liệu thực nghiệm

0.1.1 Môi trường phát triển

Hệ thống được xây dựng chủ yếu dựa trên ngôn ngữ lập trình Python, kết hợp cùng các thư viện xử lý toán học như Numpy và Pandas để thao tác với ma trận vector nhúng (Embedding Vector Matrix). Các tác vụ lớn và phức tạp trong hệ thống được định tuyến và phân rã nhờ mô hình đa tác tử (Multi-Agent System) xây dựng trên nền tảng tiến trình của thư viện *LangGraph*. Việc truy xuất (Retrieval) và sinh ngôn ngữ tự nhiên (Natural Language Generation) được thực hiện thông qua API của mô hình ngôn ngữ lớn (Large Language Model) Google Gemini 2.5 Flash, qua thư viện *google-genai*. Đối với giải thuật đo lường sự tương đồng (Similarity Search) và truy xuất dữ liệu, hệ thống tự phát triển bộ máy tìm kiếm độc lập bao gồm tìm kiếm từ khóa cục bộ (Custom BM25) và tìm kiếm ngữ nghĩa trên vector (Semantic Cosine Search) thay vì phụ thuộc vào các thư viện đóng gói sẵn thường mang lại sự cồng kềnh khó kiểm soát. Quá trình hòa trộn điểm số được thực hiện qua thuật toán RRF (Reciprocal Rank Fusion) kết hợp cùng mô hình nhúng (Embedding Model) *dangvantuan/vietnamese-document-embedding* cho ra kết quả vector 768 chiều. Nhờ thao tác tính toán trực tiếp trên ma trận Numpy thay vì cơ chế xấp xỉ của FAISS, hệ thống đạt được độ chính xác tuyệt đối (Exact Match) trong việc so khớp ngữ nghĩa. Giai đoạn tái xếp hạng (Reranking) ở bước cuối quy trình truy xuất được đảm nhiệm chuyên biệt bởi mô hình học máy Cross-Encoder *AITeamVN/Vietnamese_Reranker*. Hỗ trợ cho bộ máy trên là hạ tầng giao tiếp chuẩn hóa Model Context Protocol (MCP), giúp kết nối các tác tử AI định tuyến linh hoạt đến công cụ nội bộ lẫn bên ngoài một cách thống nhất.

0.1.2 Dữ liệu thực nghiệm

Nguồn dữ liệu gốc được khai thác từ toàn bộ tài liệu Sách Giáo Khoa Tin học lớp 10, 11 và 12, thuộc cả hai bộ sách Cánh Diều và Kết Nối Tri Thức. Qua quá trình tiền xử lý, văn bản được làm sạch và dán nhãn phân cấp từ chương, bài đến sâu nhất là tiểu mục. Tổng cộng, dữ liệu được phân rã thành 2348 khối dữ liệu (chunks) và có chứa thuộc tính level để biểu diễn độ sâu, phục vụ trực tiếp cho vòng phục hồi phân cấp (Hierarchical RAG).

0.2 Tổng quan kiến trúc toàn bộ hệ thống

Kiến trúc tổng thể của dự án được thiết kế thành một pipeline (Data Pipeline) từ bước xử lý tài liệu thô ban đầu cho tới bước sinh câu trả lời hoàn thiện phục vụ người dùng. Quá trình này được chia làm các cụm thành phần nối tiếp nhau.

Cụm Tiền xử lý dữ liệu (Data Ingestion & OCR): Nguồn dữ liệu dạng giáo trình (PDF) được nạp vào và đi qua mô hình khai phá OCR để nhận dạng, chuyển

đổi văn bản sang định dạng Markdown theo chuẩn của LlamaParse. Việc này nhằm bóc tách cấu trúc phân cấp tinh vi (chương, bài, tiểu mục) của giáo trình.

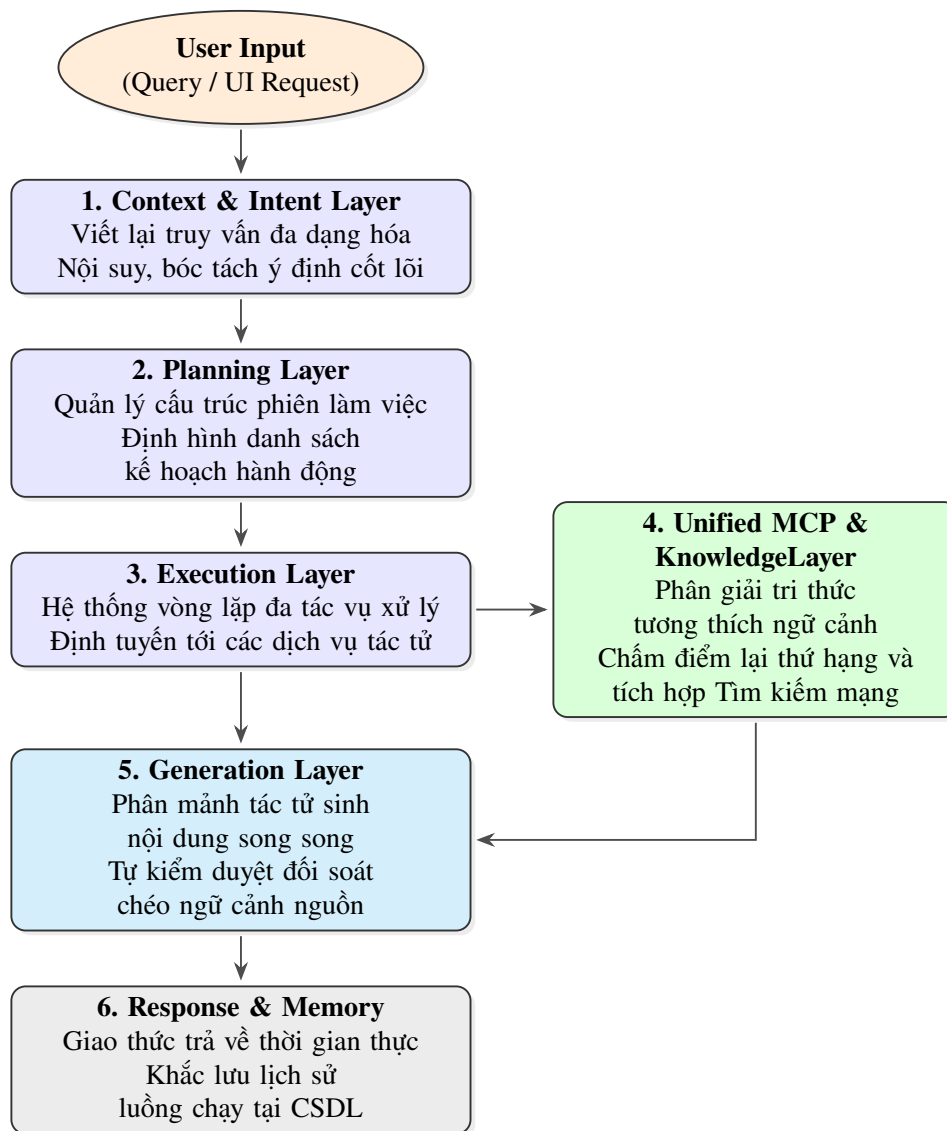
Cụm Xây dựng Cơ sở tri thức (Knowledge Base Construction): Văn bản Markdown sau quá trình làm sạch sẽ được phân rã (Chunking) theo đúng phân cấp. Mỗi khối dữ liệu (chunk) được nhúng (Embedding) thành ma trận vector và lưu trữ vĩnh viễn ở Cơ sở tri thức (Knowledge Base / Vector DB) cùng với metadata (*hệ cấp và ID cha con*).

Cụm Phân hệ Truy xuất (Retrieval & Reranking): Ở chặng run-time, hệ thống đón nhận truy vấn (User Query) từ người dùng. Cụm RAG làm nhiệm vụ định tuyến hướng đi, tiến hành tìm kiếm theo phương pháp lai ghép (Hybrid Search) phối hợp giữa phân tích từ khóa và không gian vector cùng cơ chế RRF, sau đó dùng mô hình Reranker nhằm đánh giá và rút trích các khối tri thức chứa ngữ cảnh sát nhất.

Cụm Công cụ Trung gian (MCP Tool Architecture): Đây là lớp giao tiếp chuẩn hóa mới được áp dụng, làm cầu nối giữa các Tác tử xử lý dịch vụ với các công cụ tra cứu tri thức (như Vector DB) và công cụ nền tảng web (như Web Search) nhằm đảm bảo sự liên mạch trong luồng dữ liệu.

Cụm Sinh ngôn ngữ và Khởi tạo nội dung (Generation & Multi-Agent Parallel): Nhận Top-K ngữ cảnh cô đặc ở đầu ra của cụm Truy xuất, kết hợp cùng truy vấn gốc để truyền vào mô hình ngôn ngữ. LLM sẽ không chỉ đóng vai trò phân tích nội dung, mà còn được triển khai dưới dạng kiến trúc đa tác tử song song (Fan-out async), phân chia đồng thời các nhiệm vụ chuyên biệt như sinh cấu trúc nội dung, tạo hội thoại hoặc tự động thiết kế câu hỏi luyện tập.

Sơ đồ tổng quan được biểu diễn tại Hình 0.1, mô tả toàn bộ quy trình xử lý từ khi tiếp nhận truy vấn của người dùng cho đến khi sinh ra câu trả lời cuối cùng. Kiến trúc hệ thống theo mô hình vòng đời yêu cầu được nâng cấp và tối ưu hóa thành 6 lớp xử lý liên tục: Lớp Ngữ cảnh và Ý định (Context & Intent Layer), Lớp Lập kế hoạch (Planning Layer), Lớp Thực thi (Execution Layer), Lớp Truy xuất và Giao thức Tri thức (Unified MCP & Knowledge Layer), Lớp Sinh phản hồi (Generation Layer), cùng Lớp Phản hồi và Ghi nhớ (Response & Memory). Bằng việc chia nhỏ từng chặng xử lý và ứng dụng quy chuẩn Model Context Protocol, hệ thống đảm bảo duy trì độ độc lập, dễ dàng bổ sung thêm cấu trúc công cụ mới.



Hình 0.1: Sơ đồ hệ thống tổng quan theo kiến trúc Mạng lưới Tác tử và Hệ sinh thái phân tầng

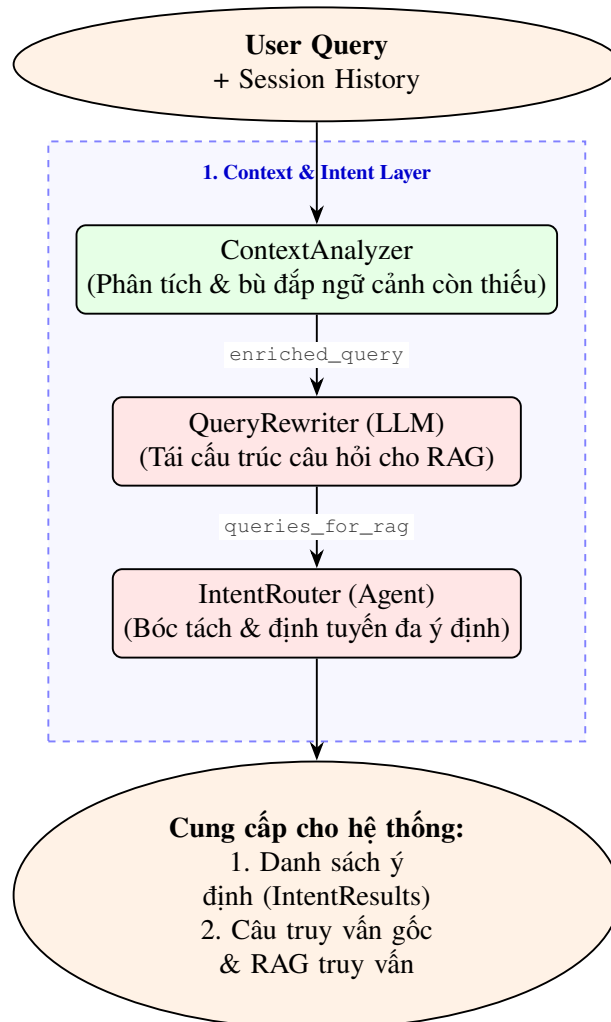
Trong Hình 0.1, kiến trúc hệ thống hoạt động thông qua một chuỗi xử lý tinh chế khép kín, trong đó Lớp số 4 đóng vai trò cốt lõi trong việc chuẩn hóa thao tác lấy thông tin gốc, chuyển đổi mọi tác vụ gọi hàm tự do thành một mạng lưới MCP có kiểm soát. Kế tiếp, năng lực giải quyết được chuyển hóa cho khối Lớp Số 5, phân rã để hoàn tất các nghiệp vụ phức tạp mà không để chậm trễ đường truyền.

0.3 Thiết kế chi tiết các thành phần hệ thống

Dựa trên sơ đồ kiến trúc tổng quát tại Hình 0.1, các thành phần cốt lõi của hệ thống được thiết kế chi tiết nhằm đảm bảo tính linh hoạt và độ chính xác cao trong quá trình xử lý:

0.3.1 Thiết kế Lớp Ngữ cảnh và Ý định (Context & Intent Layer)

Đây là lớp tiếp nhận và xử lý đầu vào đầu tiên của hệ thống, thực hiện nhiệm vụ bóc tách ngữ cảnh (ContextAnalyzer), tinh chỉnh truy vấn (QueryRewriter) và định tuyến ý định đa nhiệm (IntentRouter). Thiết kế này giúp hệ thống hiểu được không chỉ câu hỏi hiện tại mà còn cả diễn biến của toàn bộ cuộc hội thoại trước đó.



Hình 0.2: Chi tiết cấu trúc và luồng xử lý tại Lớp Ngữ cảnh và Ý định

Lớp Ngữ cảnh và Ý định hoạt động tuân theo trình tự ba bước cốt lõi:

ContextAnalyzer (Phân tích ngữ cảnh): Đánh giá mức độ phụ thuộc của truy vấn hiện tại vào lịch sử hội thoại trước đó (Session History). Mô-đun này được thiết kế bằng mã nguồn logic nhằm xác định xem câu hỏi có chứa các đại từ ngữ nghĩa ẩn ý cần khôi phục ngữ cảnh hay không.

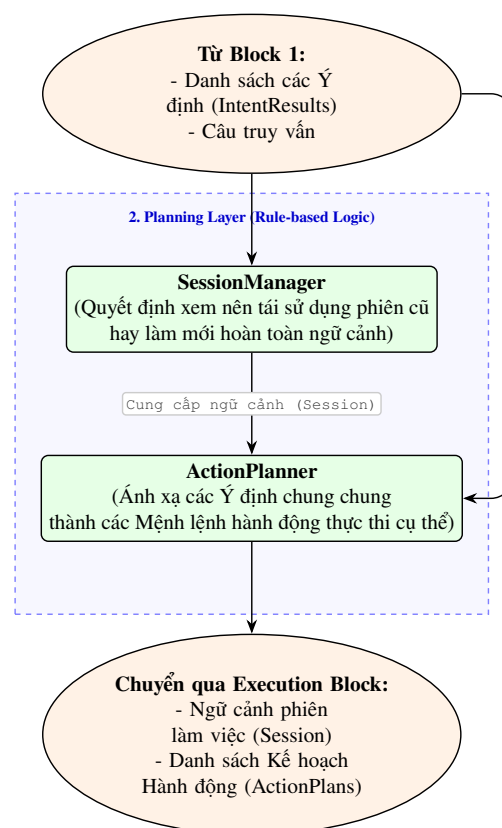
QueryRewriter (Viết lại truy vấn - LLM): Khi quá trình phân tích phát hiện truy vấn không đầy đủ ngữ nghĩa hoặc hệ thống cần mở rộng diện tìm kiếm trong RAG, LLM sẽ được gọi để sinh ra các câu hỏi hoàn chỉnh, đồng thời có thể tạo ra đa truy vấn (multi-queries).

IntentRouter (Định tuyến ý định): Đóng vai trò là thành phần định hướng cốt lõi. Thành phần này sử dụng hàm *detect_multi()* để xác định đồng thời tối đa 3 ý định (Intent) từ người dùng. Đầu ra thu được là một danh sách *List[IntentResult]* bao gồm các siêu dữ liệu quan trọng như Ý định (Intent), Loại tác vụ (Task Type), Chủ đề (Topic), Cấp bậc/Lớp (Grade) và Thứ tự ưu tiên (Order).

Nhờ kiến trúc này, hệ thống giải quyết xuất sắc các yêu cầu phức hợp (multi-intent) trong khi vẫn duy trì đúng đường định tuyến thiết kế cho các tác vụ tiếp theo.

0.3.2 Thiết kế Lớp Lập kế hoạch (Planning Layer)

Đóng vai trò như bộ não điều phối, Lớp Lập kế hoạch (Planning Layer) chịu trách nhiệm tiếp nhận và hiện thực hóa các ý định đa nhiệm thành lệnh thực thi, thay thế cho cơ chế định tuyến thủ công tĩnh thông thường. Lớp này hoạt động hoàn toàn trên phương pháp phi xác suất dựa trên luật (Rule-based Logic) thay vì LLM, giúp đảm bảo hệ thống phản hồi cực nhanh, ổn định và mang tính chất dự đoán được.



Hình 0.3: Chu trình ra quyết định và duy trì ngữ cảnh tại Lớp Lập kế hoạch (Planning Layer)

Dữ liệu đầu vào của lớp là tập hợp các *Intent* cùng câu truy vấn nguyên thủy (Query). Lớp Lập kế hoạch phân bổ chu trình qua hai thành phần cốt lõi:

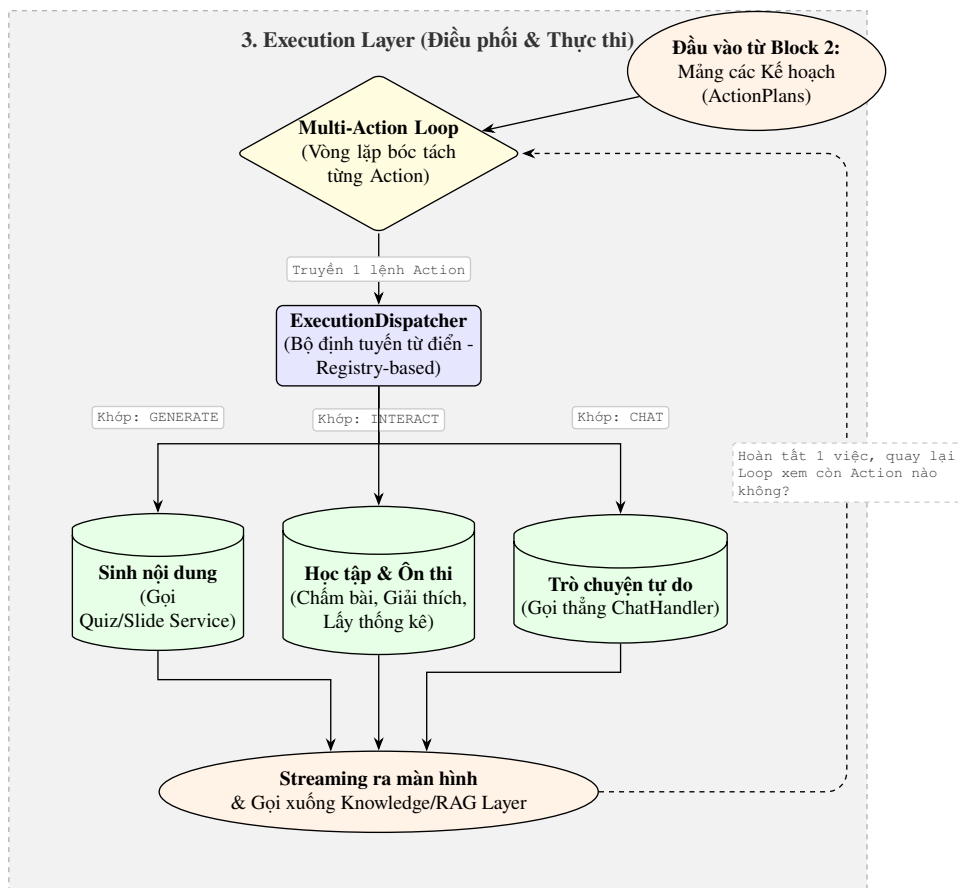
SessionManager: Nhận truy vấn và xác định người dùng đang tiếp nối một chuỗi hội thoại cũ (cần tải lại ngữ cảnh quá khứ) hay khởi tạo tác vụ mới hoàn toàn. Khối này sẽ trực tiếp duy trì không gian *Session* phù hợp.

ActionPlanner: Dựa trên cấu trúc Session nhận được từ SessionManager và các đặc trưng từ danh sách Intent, mô-đun này sẽ ánh xạ các ý định chung chung thành cấu trúc *ActionPlans* (Danh sách Lệnh hành động) rõ cấp bậc và không trùng lặp (deduplicated).

Đầu ra cuối cùng là danh sách tác vụ rành mạch, kết thúc vai trò nền tảng để đẩy sang Lớp Thực thi (Execution Layer).

0.3.3 Thiết kế Lớp Thực thi (Execution Layer)

Lớp Thực thi đóng vai trò trung tâm trong chuỗi xử lý đa nhiệm của hệ thống, có nhiệm vụ bóc tách danh sách các thao tác đã được Lập kế hoạch và phân phối chúng đến đúng bộ dịch vụ chuyên trách (Handler). Cơ chế định tuyến này giúp quy trình diễn ra liên tục, linh hoạt và độ chính xác cao.



Hình 0.4: Sơ đồ vòng lặp Đa tác vụ (Multi-Action Loop) tại Lớp Thực thi

Quá trình thực thi trong lớp này gồm các thành phần cốt lõi:

Cơ chế điều phối Đa hành động (Multi-Action Loop) tiến hành duyệt qua danh

sách các thao tác đã được chốt từ tầng Lập kế hoạch theo hình thức vòng lặp. Cụm tác vụ này đóng vai trò như một bộ đếm trạng thái, chỉ thả luồng hoàn tất khi hệ thống đã bóc tách và phân bổ triệt để tất cả các nhiệm vụ.

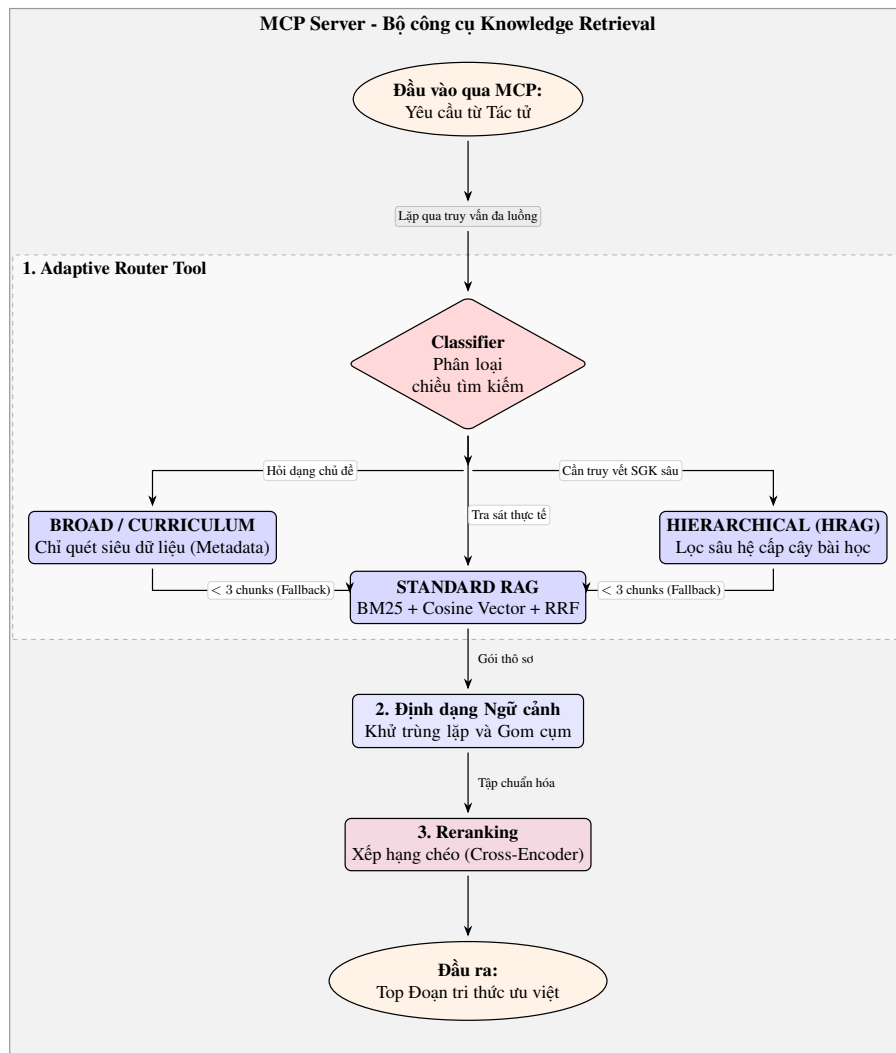
Trạm rẽ nhánh (Execution Dispatcher) dựa vào đặc tính của mỗi danh mục tác vụ để truyền tín hiệu tới Dịch vụ lĩnh vực (Domain Service) tương ứng. Lớp dịch vụ lĩnh vực là các cụm xử lý logic chuyên biệt. Đối với Hệ trợ lý học thuật này, các dịch vụ tiêu biểu có thể kể đến quá trình kiến tạo bài tập (QuizService), sinh giáo án đa phương tiện (SlideService), và cụm Handler trả lời trực tiếp. Đặc tính quan trọng nhất là các lớp dịch vụ này sẽ đóng vai trò như Tác tử Đại diện (Agent Client), thiết lập giao thức gọi Công cụ chuẩn hóa (MCP Tool Client) để liên lạc sâu xuống Lớp công cụ trung tâm của MCP Server.

Kết thúc một luồng lệnh đơn lẻ, dữ liệu sẽ được kích hoạt đẩy song song ra luồng hiển thị (streaming) ra màn hình và đồng thời lưu chuyển ngược mốc tín hiệu về vòng lặp chính để xử lý kế hoạch hành động tiếp theo.

0.3.4 Thiết kế Lớp Tri thức và Trạm Công cụ MCP (Knowledge Layer & MCP Server)

Lớp Tri thức đóng vai trò nòng cốt trong việc giải quyết các tác vụ chuyên sâu (Knowledge-Intensive Tasks) lấy trọng tâm chuẩn hóa Sách giáo khoa. Để giao tiếp với lớp này, hệ thống áp dụng kỹ thuật bọc giao diện theo quy chuẩn Model Context Protocol (MCP). Bằng cách thiết kế công cụ *KnowledgeRetrievalTool* trên một MCP Server trung tâm, mọi yêu cầu truy xuất dữ liệu từ các Tác tử cấp trên đều được thống nhất về một chuẩn duy nhất, hoàn toàn tách biệt khỏi logic luồng thực thi (Decoupled Layer).

Kiến trúc tìm kiếm RAG của hệ thống đã được tinh luyện lại thành *Advanced Adaptive RAG*. Tầng này không hoạt động trên một luồng tĩnh cố định mà sử dụng Bộ phân loại (Classifier) để tự động định hướng cách thức truy xuất linh hoạt dựa trên dạng câu hỏi. Đáng nói, tầng xử lý phía bên dưới là sự kết hợp sâu sắc của các giải thuật xây dựng thủ công. Điển hình là sự nâng cấp của Khối Lai ghép (Hybrid Searcher), trực tiếp tổng lũy số liệu từ Hàm truy vết từ khóa (Custom BM25) cùng giải thuật tương đồng Cosine của Vector ma trận thuần (Numpy Dot Product) thông qua công thức hội tụ cực trị Reciprocal Rank Fusion (RRF). Việc giải quyết toán học trực tiếp trên vi mạch Vector Numpy mang lại lợi ích bám sát tuyệt đối đáp án chính xác (Exact Match) so với các giải thuật độ xấp xỉ của các nhóm thư viện bên thứ 3.



Hình 0.5: Cấu trúc Máy chủ Công cụ MCP và Quá trình Truy xuất sâu (Knowledge Layer)

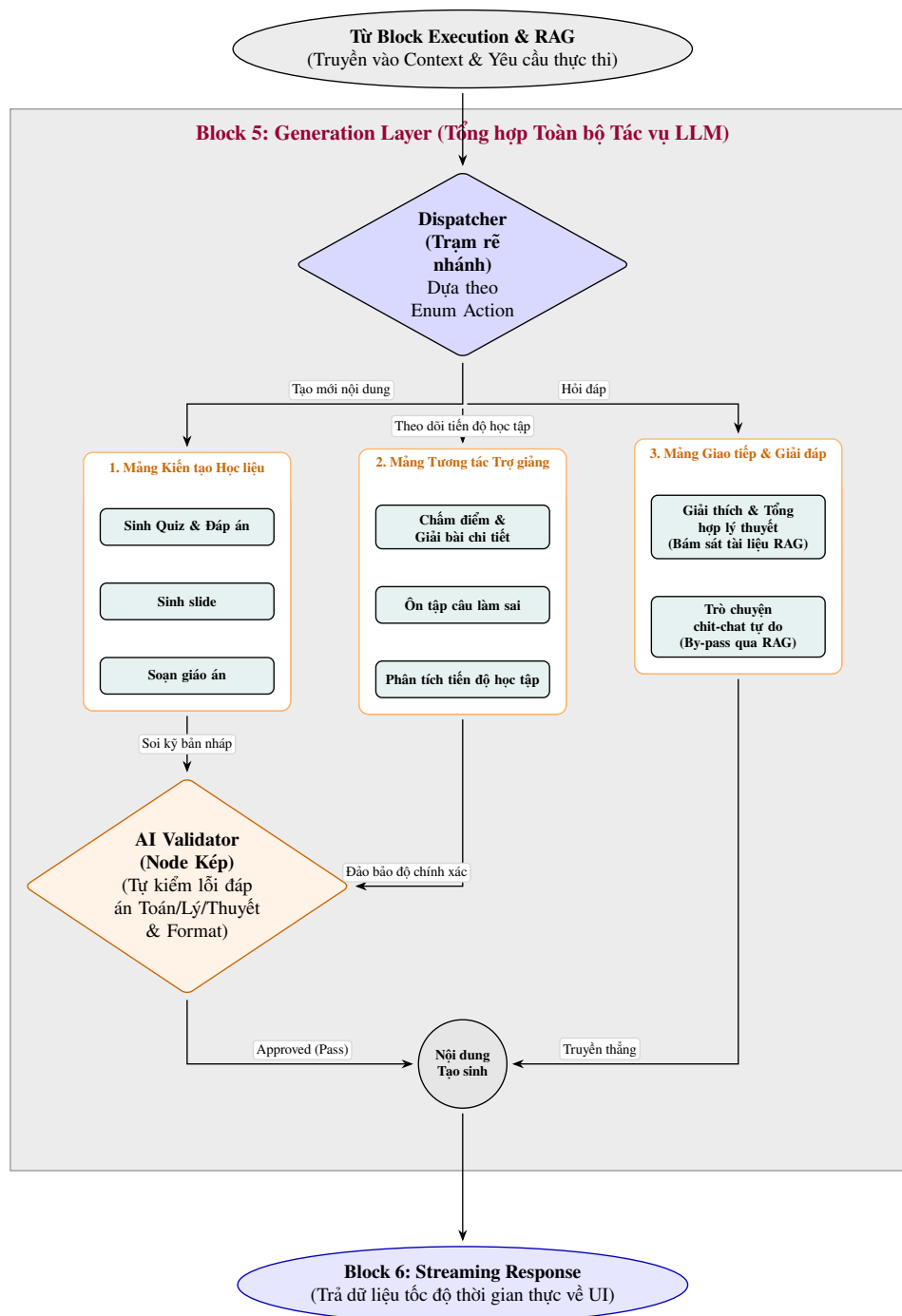
Đầu vào là tổng quát lệnh được chuyển từ máy khách vào Máy chủ MCP Tools. Hệ thống tự động phân loại mức ý định (hỏi cấu trúc, hỏi sâu, hoặc tìm kiếm quy mô rộng) để tối ưu chi phí tính toán. Cụ thể quy trình chiết xuất thông tin được thực hiện qua các giai đoạn lọc triệt để. Tuyến đường trung tâm phụ trách khâu rà quét thô bằng kỹ thuật RRF (Reciprocal Rank Fusion) để hợp nhất sức mạnh dò tìm văn bản và hình học ma trận. Rổ dữ liệu sau khi khử những đoạn trùng lặp tiếp tục trải qua mô hình đánh giá chéo máy học chéo (Cross-Encoder Reranking). Top những phân đoạn xếp hạng cao nhất được xem là nguyên liệu thuần túy nhất để nhồi vào prompt điều khiển cho khối tác tử xử lý ngôn ngữ đầu xả (Generative Generation Layer).

0.3.5 Thiết kế Lớp Sinh phản hồi và Hậu kiểm (Generation & Validation)

Lớp Sinh phản hồi đóng vai trò hợp nhất dữ liệu từ khối kiến thức và ý định người dùng để sản sinh câu trả lời cuối cùng. Để tối ưu hóa độ trễ tính toán, mô-đun phát sinh giao án và giáo cụ học tập được tái cấu trúc hoàn toàn sang mô hình

Đa tác tử song song (Fan-out async Multi-Agent Architecture). Cụ thể, khi người dùng yêu cầu soạn một bài giảng có độ dài phức hợp, một trạm điều phối sẽ đánh thức đồng thời ba tác tử phụ (Sub-agents) chạy bất đồng bộ. Nhánh thứ nhất chuyên lo về phần mở đầu và định hướng mà không cần truy xuất dữ liệu RAG. Nhánh thứ hai tập trung khai thác lượng văn bản từ SGK để biên soạn lý thuyết. Nhánh thứ ba kế thừa chức năng tạo bài tập trắc nghiệm để sinh các câu hỏi củng cố. Tính năng này giúp triệt tiêu hoàn toàn độ trễ so với việc chạy phản hồi chuỗi tuyến tính.

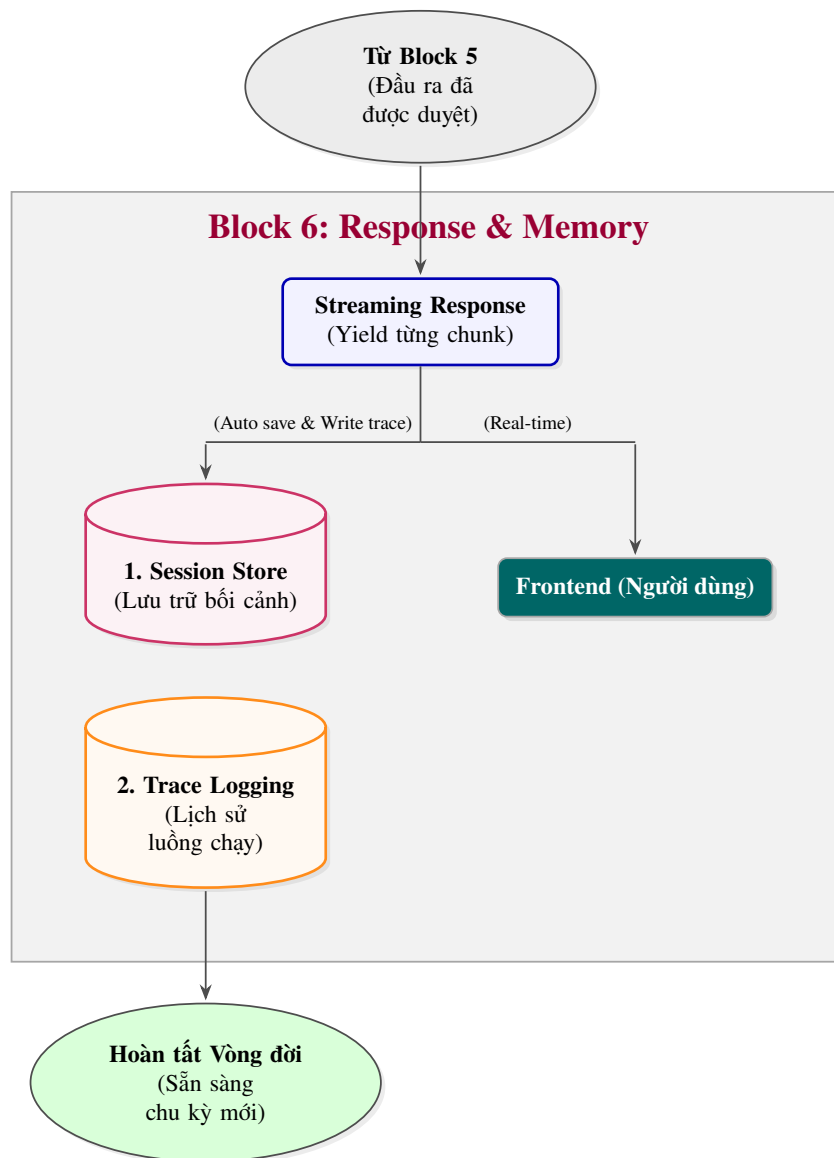
Cộng hưởng cùng quy trình song song trên, một Tác tử Đa phương tiện (Media Agent) có khả năng kích hoạt công cụ Tìm kiếm mạng lưới (Web Search Tool qua giao thức liên kết MCP) để phân tích bối cảnh bài học, sau đó tự động cào ảnh nền, biểu đồ minh họa từ Internet và chèn thẳng vào đầu ra giao diện thay vì chỉ hiển thị dạng văn bản thuần túy. Trước khi được hiển thị, mọi đầu ra đều phải vượt qua trạm Kiểm duyệt viên AI (Validator Agent) thực hiện đối soát chéo văn bản hệ thống sinh ra với nguồn tri thức cơ sở, chấm dứt hiện tượng ảo giác (hallucination) cũng như các lỗi định dạng sai lệch. Thiết kế phân luồng này được mô tả chi tiết tại Hình 0.6.



Hình 0.6: Giai đoạn phân luồng Tạo sinh và Hậu kiểm (Block 5)

0.3.6 Thiết kế Streaming Response, History Memory và Logging

Đây là chặng cuối cùng (Block 6) trong vòng đời xử lý của hệ thống, đóng vai trò như một cầu nối giao tiếp trực tiếp với người dùng và đồng thời dọn dẹp, lưu trữ trạng thái trước khi kết thúc phiên làm việc. Thiết kế của lớp này ưu tiên hai yếu tố: tốc độ hiển thị cho trải nghiệm người dùng (mượt mà thông qua Streaming) và tính toàn vẹn dữ liệu của hệ thống (lưu vết và ngữ cảnh). Sơ đồ thao tác của Block 6 được thể hiện tại Hình 0.7.



Hình 0.7: Khối thao tác Phản hồi và Khắc ghi tiến trình (Block 6)

0.4 Đánh giá hiệu năng và Lịch trình theo dõi năng lực

0.4.1 Phương pháp đánh giá và Điểm tựa hiệu năng (Baseline)

Hệ thống sử dụng phương pháp đánh giá định lượng (Quantitative Evaluation) dành riêng cho kiến trúc RAG thông qua nền tảng đo lường tự động Ragas Framework. Quy trình kiểm thử (RAGAS Pipeline) diễn ra liên tục từ bước Trình phát sinh bộ câu hỏi (TestsetGenerator), tự động sinh hàng trăm mẫu truy vấn và tham chiếu (Ground Truth) từ 2348 phân đoạn tri thức trong ngân hàng dữ liệu. Sau đó, Bộ thu thập và đánh giá (Evaluator) sẽ chấm điểm chéo dựa trên bốn hệ số thiết yếu: mức độ trung thành với tài liệu gốc (Faithfulness), sự bám sát của câu trả lời với truy vấn (Answer Relevancy), sự chuẩn xác khi trả ngữ cảnh (Context Precision) và độ bao phủ của văn bản hồi đáp (Context Recall).

Do việc xử lý tính toán liên quan đến ma trận đa chiều, thời gian phản hồi

(Latency) Baseline khi hệ thống hoạt động đầy đủ tính năng tiêu tốn xấp xỉ 21 giây trên CPU phổ thông. Phân tích nguyên lý hoạt động cho thấy công đoạn viết lại câu hỏi (Query Rewrite) mất khoảng một giây, việc tìm kiếm hồ sơ mất chưa tới 0.3 giây; tuy nhiên, mô hình chấm hạng chéo (Reranker) đang đứng điểm nghẽn (bottleneck) chiếm trọn 95% thời lượng. Nguyên do đến từ giới hạn phần cứng không tận dụng sức mạnh thẻ đồ họa tăng tốc. Bù lại, người dùng nhận được sự bù đắp nhờ chức năng xử lý đa tác vụ hiển thị Streaming theo thời gian thực.

0.4.2 Theo dõi và Lưu vết năng lực cá nhân (Student Tracking)

Bên cạnh đánh giá chất lượng mô hình, hệ thống còn tập trung cung cấp giá trị cho người học thông qua tính năng theo dõi tiến độ (Student Tracking). Mọi bài kiểm tra trắc nghiệm sinh ra đều được Hệ thống đính kèm định danh độ khó và khóa phân mảnh chủ đề. Ngay khi người dùng nộp bài, Cơ sở dữ liệu Hồ sơ Học viên (StudentProfile) sẽ tự động lưu lại các chỉ số phần trăm hoàn thiện. Theo thời gian, hệ thống có khả năng tái tạo một Bản đồ tri thức (Knowledge Map) của cá nhân, biểu thị rõ liên kết giữa các bài học mà học sinh nắm vững hay còn yếu điểm để gợi ý ôn tập chính xác, vượt qua ranh giới đánh giá thông thường của một Chatbot hỏi đáp.

Quá trình đánh giá được thực hiện trên 4 độ đo (Metrics) cốt lõi của Ragas:

Faithfulness (Độ trung thực): Đo lường mức độ câu trả lời của mô hình bám sát vào ngữ cảnh (context) mà hệ thống truy xuất được. Mức điểm cao chứng tỏ mô hình không bịa đặt thông tin (Hallucination).

Answer Relevancy (Độ phù hợp): Đánh giá xem câu trả lời có giải quyết trực tiếp truy vấn ban đầu của người dùng hay không, tránh việc trả lời lan man, sai trọng tâm.

Context Precision (Độ chính xác ngữ cảnh): Đánh giá khả năng xếp hạng của cụm truy xuất. Điểm số cao đồng nghĩa với việc các tài liệu liên quan nhất được xếp ở các thứ hạng đầu (Top-K).

Context Recall (Độ bao phủ ngữ cảnh): Đo lường khả năng hệ thống tìm ra đầy đủ các thông tin cần thiết từ Cơ sở tri thức (Knowledge Base) so với đáp án tiêu chuẩn (Reference).

0.4.3 Kết quả thực nghiệm phân hệ RAG

Sau quá trình chạy tự động 246 mẫu thử trên toàn bộ pipeline của hệ thống, kết quả điểm số đánh giá trung bình trên các độ đo như sau:

Chỉ số đánh giá (Metric)	Điểm số trung bình (Mean)
Faithfulness (Độ trung thực)	0.9757
Answer Relevancy (Độ thiết thực của câu trả lời)	0.8571
Context Precision (Độ chính xác của ngữ cảnh)	0.8555
Context Recall (Độ bao phủ của ngữ cảnh)	0.9593

Bảng 1: Kết quả đánh giá hệ thống theo framework Ragas

Phân tích kết quả:

- **Faithfulness (0.9757)** và **Context Recall (0.9593)** đạt mức điểm rất cao (trên 0.95). Điều này minh chứng cho tính hiệu quả của chiến lược Hierarchical RAG kết hợp với cơ chế Chunking theo chuẩn phân cấp SGK. Hệ thống đã trích xuất được gần như hoàn hảo các đơn vị kiến thức cần thiết mà không xuất hiện tình trạng sinh thông tin ảo (Hallucination) từ phía Generative Model (Gemini). Validator Layer ở Block 5 cũng góp phần gia tăng chỉ số trung thực này.
- **Context Precision (0.8555)** và **Answer Relevancy (0.8571)** dao động ổn định ở mức 85%. Điểm số báo hiệu sự hiệu quả của mô hình tái xếp hạng Cross-Encoder trong việc đưa các tài liệu mang nhiều ý nghĩa lên đầu bảng, giúp Prompt được nhúng với ngữ cảnh làm trọng tâm, từ đó trả lời trúng đích câu hỏi người dùng.

0.4.4 Đánh giá độ trễ hệ thống (Latency)

Bên cạnh chất lượng phản hồi, hệ thống còn được đo đạc về mặt thời gian thực thi (Latency) nhằm đánh giá trải nghiệm người dùng thực tế. Các chỉ số thời gian được ghi nhận cho mỗi truy vấn đầy đủ như sau:

Thành phần thực thi	Thời gian trung bình (Giây)
Khởi truy xuất (Retriever & Reranker)	4.579
Khởi sinh câu trả lời (LLM Generator)	1.979
Tổng thời gian Pipeline	6.558

Bảng 2: Thống kê thời gian trễ của các khối kiến trúc trong hệ thống

Tổng thời gian cho một vòng lặp (từ lúc nhập truy vấn đến khi sinh mã vạch response trả về Frontend) đạt trung bình xấp xỉ 6.5 giây. Trọng số thời gian chủ yếu nằm ở khâu Retriever (hơn 4.5 giây) do hệ thống phải thực hiện Hybrid Search

(bao gồm Dense Search kết hợp Custom BM25) sau đó cộng dồn với thời gian xử lý Reranker của Cross-Encoder để chất lọc dữ liệu. Tuy nhiên, việc áp dụng chiến lược sinh luồng trực tiếp (Streaming Response) ở Block 6 đã loại bỏ hiện tượng "đơ" giao diện, nâng cao trải nghiệm phản hồi (TTFT - Time to First Token được hạ thấp lý tưởng) và khiến người dùng cảm giác hệ thống trả lời tức thời.